

Product Requirements Document: MLP Training Dynamics Visualizer

1. Product Overview

The MLP Training Dynamics Visualizer is a web-based 2D simulation tool designed to help cognitive science students tangibly observe the internal mechanics of a Multi-Layer Perceptron (MLP). By utilizing relatable tabular datasets (e.g., the UCI Zoo database), the tool bypasses the visual clutter of image-based networks. It features a unique "glass box" approach, combining real-time node activations, static weight heatmaps, and rolling weight-delta visualizations to illustrate both the network's current state and its ongoing "effort" toward convergence.

2. Target Audience

- **Primary:** Students in introductory cognitive science or computational modeling courses learning the fundamentals of connectionism, artificial neural networks, and backpropagation.
- **Secondary:** The instructor, who requires a low-friction, flexible backend to configure simulation parameters and swap datasets between lectures.

3. Core Features & Requirements

3.1. Teacher Configuration & Backend

The system must allow the instructor to dictate the dataset and initial network architecture without requiring a complex database or authenticated admin dashboard.

- **Configuration Method:** A simple `config.json` or `config.yaml` file located in the backend directory. The web app reads this configuration on startup.
- **Configurable Parameters:**
 - Dataset file path (e.g., `data/zoo.csv`).
 - Feature mapping (defining which columns serve as input nodes and which column is the target label).
 - Default network architecture (e.g., `[15, 12, 12, 5]` representing Inputs -> Hidden 1 -> Hidden 2 -> Outputs).
- **Flexibility:** The instructor can quickly swap datasets by editing the configuration file and placing a new CSV into the data folder before a session.

3.2. Live Sample & Activation Visualization (The "Glass Box")

To help students map input data directly to internal network behavior, the simulation must visually broadcast the forward pass of individual data samples.

- **Current Sample Display:** A dedicated UI panel displaying the exact row of data currently passing through the network. Features a readable table showing Feature Names -> Current Values, alongside the Target Label.
- **Dynamic Node Activations:** Every node in the network acts as an indicator light.
 - **Inputs:** Fill color correlates to the normalized feature value (e.g., dark for 0, bright for 1).
 - **Hidden Layers:** Colored based on the output of their activation function (representing how strongly the neuron fired).
 - **Outputs:** Colored to represent the network's confidence/probability for each class.
- **Pacing Controls:** To prevent nodes from flickering too rapidly for the human eye, the UI must include a **"Playback Speed"** slider and a manual **"Step-by-Step"** button to allow the class to observe specific inputs.

3.3. The Dual-Network Edge Visualization

The network's edges (connections) will be displayed in two side-by-side graphs to differentiate between current learned values and the rate of learning.

- **Graph 1 (Current Weights):** Visualized using an fMRI-style diverging color map. Edges are colored on a spectrum (e.g., cool colors for negative weights, warm colors for positive weights, fading to a transparent neutral for near-zero).
- **Graph 2 (Weight Deltas):** Displays the rolling average of the change in weight over the last N gradient descent steps. Uses a sequential color map (e.g., bright yellow/white for high change, fading to dark purple/black for zero change) to visually indicate when the network has converged and stopped "learning."

3.4. Error Representation & Loss Functions

Students need to see a real-time representation of the network's performance, mapped alongside its visual learning effort.

- **Visualizing Error:** A live, updating line chart displaying the Loss Curve over epochs.
- **Loss Function Toggle:** A dropdown menu allowing the user to select the mathematical definition of the error before initializing training:
 - **Cross-Entropy Loss:** For classification tasks.
 - **Mean Squared Error (MSE):** For regression or simplified binary tasks.
 - **Mean Absolute Error (MAE):** To demonstrate a different gradient slope.

3.5. Gradient Descent Modes

To teach the mechanics of optimization and the impact of batching, the UI must include a toggle to switch processing methods.

- **Batch Gradient Descent:** Calculates error across the entire dataset before updating weights. Visual result: The delta network flashes smoothly, and the loss curve is stable. (Note: For visual node activations during batch mode, the UI should allow the instructor to "ping" the network with a specific row to see the pathways without altering the batch training math).
- **Stochastic Gradient Descent (SGD):** Updates weights after evaluating a single instance. Visual result: The delta network flashes more chaotically, the nodes light up sequentially sample-by-sample, and the loss curve exhibits high variance.

3.6. Export and Snapshotting

- **Snapshot Button:** A utility button located near the visualizations.
- **Functionality:** Packages the current state of the Dual Network Graph (Weights and Deltas), the currently active data sample, and the Loss Curve into a clean PNG or JPG image file for student assignments or instructor slides.

4. Technical Stack Recommendation

- **Frontend & UI Framework: Streamlit.** Handles sliders, dropdowns, and layout natively using pure Python, allowing for rapid development.
- **Graph Visualization: NetworkX** (for calculating the 2D spatial layout of nodes and edges) mapped into **Plotly** or **Matplotlib** (to render the interactive, dynamic colors and export the final frames).
- **Machine Learning Backend: PyTorch** or **NumPy**. Leveraging standard Python data science libraries ensures the backpropagation math is accurate and easily extensible if new loss functions or activation types are needed later.

5. UI Layout Structure

The interface utilizes a structured, wide-screen grid approach:

- **Left Sidebar (Simulation Controls):**
 - Play / Pause / Step-by-Step execution controls.
 - Playback Speed / Slow-Motion slider.
 - Learning Rate slider.
 - Dropdowns for Loss Function and Optimization Mode (SGD/Batch).
- **Left Panel (Data Feed):**
 - The **Current Sample Display**, showing the active row's features and target label. Positioned adjacent to the input nodes of the network graph so the user's eye tracks left-to-right.
- **Main Canvas Top (The Networks):**
 - **Left Graph:** Absolute Weights (colored edges) + Node Activations (flashing nodes).
 - **Right Graph:** Weight Deltas (the rolling average "effort" visualization).
- **Main Canvas Bottom (Analytics):**
 - The live-updating Loss Curve graph.
 - The Snapshot export button.